

PROJECT REPORT

Selenium Python Framework Development Using Selenium, Python and Pytest

Name-Senjuti Khan

Enrollment No-12023002001154

Department - CSE

Abstract

Manual testing does not scale. Every time a web application changes even slightly, someone has to click through the same login forms, search boxes and checkout flows all over again, and sooner or later something gets missed. This project was built to take that repetitive, error-prone work off a tester's plate. It is a UI automation framework written in Python, driven by Selenium WebDriver and organised around Pytest, that lets a small team write and maintain browser tests without the usual headaches of flaky waits, duplicated locators, and unreadable results. The framework follows the Page Object Model (POM) design pattern, separates test data from test logic through CSV-driven parametrization, and automatically produces an HTML report with embedded screenshots whenever a test fails. The result is something closer to a real internal QA tool than a classroom demo — it is meant to be extended, plugged into a CI pipeline, and handed off to someone else without a long explanation.

1. Introduction

Web applications today ship changes frequently — a new release can go out weekly, sometimes daily. Every release carries a risk of breaking something that used to work: a login form, a search filter, a checkout button. Regression testing is the safety net that catches this, but doing it by hand does not hold up once an application grows past a handful of pages. Someone has to repeat the same clicks, the same form fills, the same checks, again and again, and human attention is exactly the resource that runs out first in that kind of work.

Test automation exists to solve this, and Selenium WebDriver has become the de facto standard for driving real browsers programmatically. On its own, though, Selenium is just a low-level browser control library — it does not tell you how to structure a project, how to avoid duplicating the same locator in twenty different test files, or how to keep a test suite readable as it grows to hundreds of cases. That structural layer is what this project sets out to build: a framework, not just a collection of scripts.

2. Problem Statement

Teams that automate their UI tests without a clear structure tend to run into the same handful of problems, usually in this order:

- **Locators scattered everywhere.** Element IDs and XPath expressions get hard-coded directly inside test functions, so a single UI change — a renamed button, a moved field — forces edits across dozens of files.

- **Flaky tests.** Tests that pass locally and fail on the build server, or fail one run in five, because the script tried to interact with an element before the page had actually finished rendering it.
- **Hard-coded test data.** Login credentials and search terms typed straight into the test code, which means adding one more test scenario means writing one more near-duplicate test function.
- **No visibility into failures.** A failed test just says “AssertionError” in a terminal log, with no screenshot, no context, and no easy way to tell a real bug apart from a timing glitch.

None of these problems are exotic — they are exactly what shows up in most first attempts at UI automation. The aim of this project was to solve all four at once, with a structure that a new team member could understand in an afternoon rather than a week.

3. Objectives

- Build a reusable, maintainable automation framework around the Page Object Model, so UI locators live in exactly one place.
- Eliminate flaky, timing-related failures using explicit waits instead of fixed `sleep()` calls or unreliable implicit waits.
- Separate test data from test logic through CSV-driven, data-driven testing, so new scenarios don’t require new code.
- Automate browser and driver setup across Chrome and Firefox without manual driver-binary management.
- Produce a self-contained HTML report after every run, with screenshots automatically attached to failed tests.
- Keep the whole thing simple enough to plug into a CI/CD pipeline later without rework.

4. Proposed Solution

The framework is built around the Page Object Model, which is really just a discipline: every web page (or major component of a page) gets its own Python class, and that class owns two things — the locators for its elements, and the actions a user can perform on it. Test files never talk to Selenium directly; they talk to page objects. When the UI changes, the fix happens in one page class instead of in every test that happens to touch that page.

Sitting underneath every page class is a `BasePage`, which wraps raw Selenium calls in explicit `WebDriverWait` conditions. Instead of clicking an element and hoping it’s ready, `click()` waits until the element is actually clickable; instead of asserting visibility and risking an exception, `is_displayed()` catches the failure case and returns a plain boolean. This one design decision removes most of the flakiness that automation projects usually fight for months.

Test data lives outside the code entirely, in CSV files under `data/`, and a small utility parses those files into tuples that Pytest’s `@pytest.mark.parametrize` consumes directly. Adding a new login scenario — an empty password, a locked account, a valid combination — becomes a one-line addition to a spreadsheet, not a new function.

Finally, a `DriverFactory` utility uses `webdriver-manager` to fetch the correct ChromeDriver or GeckoDriver binary automatically, and a Pytest hook (`pytest_runtest_makereport`) watches

every test's outcome, taking a screenshot the instant something fails and embedding it directly into the generated report. Nobody has to dig through a screenshots folder trying to match a filename to a failure.

4.1 Project Architecture

The codebase is organised into clearly separated layers, each with one job:

Layer / Folder	Responsibility
pages/	Page Object classes (BasePage , HomePage , LoginPage) — locators and UI actions.
tests/	Test cases and assertions (test_login.py , test_search.py).
data/	CSV files driving parametrized, data-driven tests.
utils/	DriverFactory , ConfigReader , CSV parsing helpers.
conftest.py	Shared Pytest fixtures and the failure-screenshot hook.
reports/ & screenshots/	Generated HTML reports and failure evidence.

5. Tools and Technologies Used

Component	Technology
Programming Language	Python 3
Browser Automation	Selenium WebDriver 4.x
Test Runner	Pytest
Reporting	pytest-html (self-contained HTML reports)
Driver Management	webdriver-manager
Data-Driven Testing	CSV files + Pytest parametrize
Design Pattern	Page Object Model (POM)
IDE	PyCharm

6. Working / Execution Flow

1. A test run begins with a single command: **pytest**, picked up automatically by the settings in **pytest.ini**.
2. The driver fixture in **conftest.py** launches a browser through **DriverFactory** and navigates to the configured base URL.
3. Each test interacts with the application only through page objects — never through raw Selenium calls.
4. For login tests, Pytest reads every row of **login_data.csv** and runs the same test logic once per row.
5. If an assertion fails, the **makereport** hook captures a screenshot and embeds it in the report before the browser closes.
6. Once the suite finishes, the driver fixture calls **driver.quit()**, and a complete **report.html** is ready to open.

7. Impact and Benefits

The most immediate impact is time. What used to be an afternoon of manually clicking through login and search flows now runs unattended in a few minutes, and it runs the same way every single time — no missed steps, no fatigue-driven mistakes. Because locators are centralised in page classes, a UI redesign that would once have meant hunting through dozens of test files now means editing one class.

There’s a quieter benefit too, which is trust. A test suite that fails randomly gets ignored within a few weeks — people start assuming the failure is “probably just Selenium being flaky” and stop looking. Because this framework leans on explicit waits and produces a screenshot the moment something actually breaks, failures start meaning something again. That, more than the automation itself, is what makes a framework worth maintaining long-term.

- Faster regression cycles, especially before releases.
- Lower maintenance cost when the UI changes, since locators exist in one place.
- Test data anyone can edit — including someone who has never opened the code.
- Visual proof of failure via automatic screenshots, which shortens debugging time considerably.
- A structure that scales — adding a new page or module doesn’t require rethinking the framework.

8. Limitations and Drawbacks

While the framework solves key maintenance and reliability challenges, several constraints remain:

- **UI-Only Scope:** Tests validate presentation-layer behavior without direct verification of backend APIs or database states[cite: 2].
- **Sequential Execution:** Tests currently execute sequentially, increasing total suite execution duration as test volume grows[cite: 2].
- **Local Environment Dependency:** Execution is restricted to local hardware, lacking distributed execution or cloud grid infrastructure[cite: 2].
- **No Automated Retries:** Lacks built-in retry mechanisms for transient environmental interruptions or network latency spikes[cite: 2].

9. Future Scope

To expand scalability and enterprise readiness, upcoming iterations will focus on:

- **CI/CD Integration:** Incorporating automated pipeline execution via GitHub Actions or Jenkins on pull requests[cite: 2].
- **Parallel & Cloud Execution:** Enabling `pytest-xdist` for multi-threaded runs and integrating cloud grids like BrowserStack[cite: 2].
- **Hybrid API Validation:** Augmenting UI test flows with direct REST API assertions to catch underlying service errors[cite: 2].
- **Containerization & Retries:** Packaging the execution suite in Docker containers and integrating transient failure retry plugins[cite: 2].

10. Conclusion

This project set out to solve a fairly ordinary problem — the fact that manual UI testing doesn’t scale — with a fairly disciplined answer: separate what changes often (locators, test data) from what stays stable (test logic), and make failures visible instead of cryptic. The Page Object Model keeps the codebase maintainable, explicit waits keep it reliable, CSV-driven data keeps it flexible, and the automatic screenshot-on-failure hook keeps it honest. None of these ideas are new on their own; the value here is in putting them together into something a team could actually adopt, extend, and trust — which is, in the end, the real measure of whether a testing framework is worth having.

References

- Selenium WebDriver Documentation — <https://www.selenium.dev/documentation/>
- Pytest Documentation — <https://docs.pytest.org/>
- pytest-html Plugin — <https://pytest-html.readthedocs.io/>
- webdriver-manager (PyPI) — <https://pypi.org/project/webdriver-manager/>
- Martin Fowler, “PageObject” — <https://martinfowler.com/bliki/PageObject.html>